

应用架构说明

本应用采用鸿蒙 ArkUI 声明式开发范式，架构设计遵循 UI 与逻辑分离的原则（尽管在小型应用中它们物理上位于同一文件中，但在逻辑上是分层的），并结合 单向数据流 和 持久化存储 机制。

逻辑分层架构

应用整体分为三层架构：

表现层:负责界面的渲染与用户交互。主要组件包括：顶部信息栏（分数/标题）、中部游戏区（题目/输入框/反馈）、底部控制区（开始/停止按钮）。
技术实现:使用 Column, Row, Text, TextInput, Button 等基础组件，通过 @Builder 装饰器将不同区域拆分为独立的构造函数 (HeaderSection, GameAreaSection, ControlSection)，提高代码可读性。

状态管理层:负责驱动 UI 变化的数据模型。UI 状态 (@State): 管理生命周期内的临时数据，如 currentScore（当前得分）、questionText（题目文本）、isGameRunning（游戏运行状态）。全局持久化状态 (PersistentStorage + @StorageLink): 管理跨应用周期的关键数据，即 highScore（历史最高分）。当该变量改变时，系统底层自动将其写入磁盘。

业务逻辑层:负责核心算法运算。包含题目生成算法、自定义四则运算解析器、答案校验逻辑。实现方式：封装在 Index 组件内部的私有方法中 (generateNextQuestion, calculateExpression)，不依赖 UI 组件，仅对数据负责。

核心功能实现思路

题目生成功能

题目生成必须满足以下严格约束：

1. 数字范围: 0-100。
2. 复杂度: 3-5 个运算符（即 4-6 个操作数）。
3. 多样性: 至少包含 2 种不同运算符。
4. 适用性: 针对小学生，要求结果必须为非负整数，且过程中不能出现除数为 0。

实现算法流程：

1. 循环尝试机制：采用 `do-while` 或 `while` 循环结构。如果生成的题目计算结果不合法（如小数、负数），则丢弃该题，重新生成，直到获得合法题目。
2. 随机化参数：`opsCount`: `Math.random()` 生成 3 到 5 的整数。`numbers`: 循环生成 `opsCount + 1` 个 `[0, 100]` 的整数。`operators`: 循环生成 `opsCount` 个运算符（+, -, *, /）。
3. 运算符多样性检查：使用 `Set<string>` 数据结构存储生成的运算符。判断 `Set.size` 是否大于等于 2。若不足，直接 `continue` 跳过本次循环。
4. 合法性预计算：在显示题目给用户之前，先调用内部的计算引擎算出结果。校验条件：`result >= 0` 且 `Number.isInteger(result)`。

由于 ArkTS 在严格模式下不建议使用 `eval()`，且为了保证类型安全，应用内置了一个简单的双通（Two-Pass）计算算法来处理优先级。

实现逻辑：

输入：数字数组 `nums`，运算符数组 `ops`。

第一轮遍历（处理乘除）：遍历运算符数组。如果遇到 * 或 /，立即取出该运算符前后的两个数字进行计算。除零保护：若遇到 / 且除数为 0，返回 `null` 标记题目无效。计算结果替换原数组中的两个操作数，并从运算符数组中移除该符号。调整索引 `i`，确保连续的乘除法能被正确处理。

第二轮遍历（处理加减）：经过第一轮后，剩余的只有 + 和 -，且数字数量比运算符多 1。按顺序从左到右累加或累减即可得到最终结果。

采用简单的有限状态机思想管理游戏流程。

状态变量定义：

`isGameRunning`: `boolean`: 控制游戏是否进行中。

`true`: 激活输入框，显示“停止”按钮，允许提交。

`false`: 禁用输入框，显示“开始”按钮，显示结算信息。

交互流程：

开始：点击按钮 -> 重置当前分 -> `isGameRunning = true` -> 生成第一题。

提交答案：

获取用户输入 `TextInput`。

数值比对：将用户输入转为 `number`，与预存的 `currentAnswer` 对比。

正确：`currentScore++` -> 生成下一题 -> 提示“正确”。

错误：`isGameRunning = false` -> 触发结算 -> 提示“错误及正确答案”。

结算逻辑：

比较 `currentScore` 与 `highScore`。若当前分更高，更新 `highScore`（系统自动持久化）。

运行截图





